Breaking the Memory and Communication Walls of Distributed Complex GEMM on Multi-Zone Heterogeneous DSP Supercomputers

Anonymous Author(s)
Double-Blind Peer Review Submission

***Abstract*—Double-precision complex general matrix-matrix multiplication (ZGEMM) is a foundational kernel in quantum mechanics, computational electromagnetics, and complex dense linear algebra solvers. Deploying large-scale ZGEMM on multi-zone heterogeneous supercomputers—exemplified by the MT-3000 processor, which couples 16 ARMv8 cores with four autonomous 24-core bare-metal VLIW/SIMD digital signal processing (DSP) clusters per node—exposes two severe bottlenecks: on-chip scratchpad bandwidth starvation without hardware data caches, and inter-node MPI collective storms across thousands of accelerator processes.**

This paper presents an end-to-end hierarchical co-design framework that breaks both walls simultaneously. At the inter-node level, we design a Stationary-A, Panel-Broadcast-B 2D hybrid orchestration ($P/2 \times 2$) featuring zero-copy strided panel extraction via committed `MPI_Type_vector` types, two-phase non-blocking subgroup collectives, offset-addressed parallel POSIX `pread` directly into DSP-mapped memory overlapped with network transfers, and 64-column vector-aligned intra-cluster partitioning. At the intra-chip level, we architect a 14-channel asynchronous direct memory access (DMA) tri-buffered scratchpad pipeline (C-matrix cyclic triple buffering nested around A/B ping-pong double buffering) paired with a 4×4 register-resident dual-FMA micro-kernel that pins 32 physical 16-lane FP64 vector accumulators, eliminating 86.1% ($7.20\times$) of Array Memory vector load/store traffic. Across intra-node benchmarks ($N = 512$ to 16384), a single DSP core sustains 49.19 GFLOPS (64.1% of dual-FMA peak, $1.42\times$ over single-buffered DMA, and $6.12\times$–$7.28\times$ over 1-core OpenBLAS), one 24-core cluster reaches 243.44 GFLOPS ($\sim 100\%$ of the 240.5 GFLOPS DDR4 DMA Roofline; $9.33\times$ over 4-core CPU and up to $6.57\times$ over 16-core OpenBLAS), and one 96-core node sustains 956.17 GFLOPS with FP64 ℓ_∞ error $\leq 2.18 \times 10^{-8}$. Scaling across 512 MT-3000 supercomputer nodes (2,048 MPI processes, 49,152 DSP cores, and 786,432 VPU lanes), our framework slashes 8192^3 collective latency from 102.25 s to 10.32 s ($9.90\times$ comm and $9.86\times$ end-to-end speedup), accelerates member parallel I/O by up to $93.5\times$, halves the 8192^3 DDR-streaming floor to 89.92 TFLOPS ($2.01\times$ over 1D), scales extreme 16384^3 workloads to 218.79 TFLOPS ($222.49\times$ speedup, 112.80% 2-node superlinear efficiency), and sustains 92.21% weak-scaling efficiency at 49,152 DSP cores.

***Index Terms*—Complex matrix multiplication (ZGEMM), multi-zone heterogeneous processor, MT-3000 DSP, software-managed scratchpad, asynchronous DMA triple buffering, 2D hybrid MPI collectives, Roofline performance model.**

I. Introduction

Double-precision complex general matrix-matrix multiplication (ZGEMM), defined for split-complex matrices $A \in \mathbb{C}^{M\times K}$, $B \in \mathbb{C}^{K\times N}$, and $C \in \mathbb{C}^{M\times N}$ as $C \leftarrow C + AB$, sits at the core of large-scale scientific simulations, including plane-wave density functional theory, frequency-domain computational electromagnetics, quantum chromodynamics, synthetic aperture radar imaging, and Complex High-Performance LINPACK (HPL) [1], [2]. Unlike real DGEMM ($2MNK$ FLOPs), ZGEMM executes $8MNK$ double-precision operations—four real matrix products and four real matrix additions/subtractions—imposing quadrupled arithmetic density and doubled memory footprints.

To meet exascale energy-efficiency mandates, modern supercomputers increasingly adopt heterogeneous many-core processors that replace hardware caches with software-managed scratchpad memories (SPMs) and explicit direct memory access (DMA) engines [3]–[6]. A prominent representative is the **MT-3000** multi-zone processor [7], coupling a 16-core ARMv8 General-Purpose (GP) Zone with a 96-core Acceleration (ACC) Zone partitioned into four autonomous 24-core bare-metal VLIW/SIMD DSP clusters (1,536 FP64 vector lanes and 11.6 TFLOPS node peak at 1.2 GHz). Each DSP core accesses a 64 KB Scalar Memory (SM) and 768 KB vector Array Memory (AM) without hardware L1/L2 caches between AM and off-chip DDR4 [8], [9].

While recent studies have optimized single-node or single-cluster real-valued operators on MT-3000 [8]–[11], scaling complex ZGEMM across hundreds of multi-zone nodes (tens of thousands of DSP cores) uncovers four coupled challenges:

1) **Complex Register Pressure and AM Traffic Amplification:** In split-complex planar storage, updating real (C_{Re}) and imaginary (C_{Im}) tiles involves four coupled cross-terms. Naive inner-loop accumulation loads and stores C_{Re} and C_{Im} from AM at every reduction step (2,304 vector memory ops per $4 \times 64 \times 32$ block), starving dual-issue fused multiply-add (FMA) units, whereas algebraic 3M reductions [2] consume extra packing workspaces and suffer $\mathcal{O}(\kappa)$ numerical cancellation [12].
2) **Asymmetric Scratchpad Capacity and Exposed C-Tile DMA:** Without hardware caches, all active A, B, and C buffers must co-reside inside 64 KB SM and 768 KB AM. Standard A/B ping-pong double buffering hides inner K-loop fetches but leaves bidirectional DDR4↔AM load and write-back latencies of C tiles exposed at outer tile boundaries.
3) **Supercomputer-Scale MPI Storms and Serialized Root I/O:** Mapping one MPI rank per autonomous DSP cluster

yields 2,048 ranks across 512 nodes. Globally broadcasting a 1.074 GB complex B matrix ($8192^2 \times 16$ B) triggers severe network congestion (102.25 s at 512 nodes—over 1,040× longer than kernel execution), while root-serialized file reading leaves 2,047 clusters idle.

4) **Strong-Scaling Off-Chip DDR-Streaming Floor:** Under strong scaling ($M = N = K = 8192$), once per-cluster row height M_{local} drops below scratchpad tile height $M_b = 56$ ($P > 146$ in 1D), operational arithmetic intensity collapses from 14.83 to 1.88 FLOPs/Byte, locking 1D execution to a hard DDR4 streaming floor (0.0982 s, 44.78 TFLOPS).

To overcome these walls simultaneously, we present a full-stack co-design spanning parallel I/O, MPI collective topology, vector alignment, multi-tier DMA pipelining, and register allocation:

- **Stationary-A, Panel-Broadcast-B 2D Hybrid Orchestration ($P/2 \times 2$):** We eliminate network transfers of A and post-compute reductions of C via disjoint 2D output ownership, zero-copy strided panel dispatch via committed `MPI_Type_vector` types, two-phase non-blocking subgroup collectives (`MPI_Isend`/`Irecv` + `MPI_Ibcast`) overlapped with concurrent offset-addressed POSIX `pread`, and 64-column vector-aligned intra-cluster partitioning (`chunk_64`), proving analytically why $G = 2$ uniquely preserves 24-core occupancy and 3-stage C-pipeline rotation at $N = 8192$.
- **14-Channel Tri-Buffered Scratchpad Pipeline and 4×4 Register-Resident Micro-Kernel:** We coordinate 14 hardware P2P DMA channels to nest inner-loop A/B ping-pong double buffering (56 KB SM + 64 KB AM) inside middle-loop C-matrix cyclic triple buffering (168 KB AM). Our 4×4 micro-kernel pins 32 physical 16-lane FP64 vector registers across $K_b = 32$ steps using hoisted SPU negation and `vec_svbcast`, cutting AM vector load/store traffic by 86.1% (7.20×).
- **First-Principles Roofline and Strong-Scaling DDR-Floor Models:** We establish a closed-form scratchpad capacity and operational Roofline model proving that tile $(56, 32, 64)$ reaches $\mathcal{I}_{\text{DMA}} = 14.83$ FLOPs/Byte (240.5 GFLOPS DDR ceiling), and derive the strong-scaling DDR-streaming floor theorem explaining why $G = 2$ yields a 2.0× asymptotic kernel speedup at extreme node counts.
- **Comprehensive Intra-Node, Ablation, and 512-Node Multi-Scale Evaluation:** Across $N = 512$ to 16384, our kernel sustains 49.19 GFLOPS on 1 DSP core (1.42× async DMA speedup; 6.12×–7.28× over 1-core OpenBLAS), 243.44 GFLOPS on 1 cluster (∼100% DDR Roofline; 9.33× over 4-core CPU), and 956.17 GFLOPS on 1 full 96-core node. Across 1 to 512 MT-3000 nodes (49,152 DSP cores), our co-design cuts 8192^3 collective time by 9.90× (102.25 s → 10.32 s) to reach 89.92 TFLOPS (2.01× over 1D), scales extreme 16384^3 workloads to 218.79 TFLOPS (222.49× speedup, 112.80% 2-node superlinear efficiency), and sustains 92.21% weak-scaling efficiency (67.24 TFLOPS, +2.77% tri-buffer jitter gain) at 512 nodes.

TABLE I
VERIFIED ARCHITECTURAL PARAMETERS OF AN MT-3000 COMPUTE NODE [7], [9]

Zone / Tier	Parameter	Verified Specification
GP Zone	Host CPU Cores	16 ARMv8 Cores (Linux OS)
	Rank Mapping	4 MPI Ranks/Node (4 GP Cores/Rank)
ACC Zone	DSP Clusters	4 Autonomous Clusters/Node (1/Rank)
	Cores / Cluster	24 Bare-Metal VLIW/SIMD Arrays
	Total Node DSPs	96 DSP Cores (1,536 FP64 VPU Lanes)
	Clock Frequency	1.2 GHz
	Dual-FMA Peak	76.8 GF/s per Core; 1.84 TF/s per Cluster
	Chip-Level Peak	11.6 TFLOPS FP64
4-Tier Memory	Vector Regs (VR)	64 × 1,024-bit (16× FP64) / Core
	Scalar Mem (SM)	64 KB / Core (SPU + `vec_svbcast`)
	Array Mem (AM)	768 KB / Core (∼921.6 GB/s Vector BW)
	GSM / HBSM	6 MB GSM/Cluster; 48 MB Shared HBSM
	Off-Chip DDR4	32 GB Shared; **16.22 GB/s** Cluster DMA

II. ARCHITECTURAL BACKGROUND AND MOTIVATION

A. MT-3000 Multi-Zone Architecture and Memory Hierarchy

As illustrated in Fig. 1 and detailed in Table I, the MT-3000 processor adopts a heterogeneous multi-zone System-on-Chip (SoC) architecture [7]. The *GP Zone* comprises 16 cache-coherent ARMv8 CPU cores running Linux, network stacks, and the MPI runtime. The *ACC Zone* integrates four physically autonomous DSP clusters managed via the lightweight `Hthread` heterogeneous driver. Because each DSP cluster maintains independent command queues, on-chip shared memories, and DMA controllers without inter-cluster cache coherence, we bind one MPI rank exclusively to each DSP cluster ($\text{dev_id} = \text{rank} \bmod 4$, four MPI ranks per physical node), eliminating intra-node device contention.

Each DSP cluster houses 24 bare-metal DSP acceleration cores clocked at 1.2 GHz. Within every DSP core, a Scalar Processing Unit (SPU) handles control flow, address generation, and scalar loads from its private 64 KB *Scalar Memory (SM)*, broadcasting scalar operands into a Scalar-Vector Register (SVR) bridge via `vec_svbcast`. Coupled to the SPU is a 16-lane FP64 Vector Processing Unit (VPU) equipped with 64 physical 1,024-bit vector registers (16×64-bit doubles per register, `lvector double`) and dual-issue FMA pipelines (`vec_mula`), delivering a theoretical vector FMA ceiling of:

$$\mathcal{P}_{\text{core}} = 16 \times 2\,\text{FMA} \times 2\,\text{FLOPs} \times 1.2\,\text{GHz} = 76.8\,\text{GFLOPS}, \quad (1)$$

or $\mathcal{P}_{\text{cluster}} = 24 \times 76.8 = 1{,}843.2$ GFLOPS per 24-core cluster [8]. The VPU executes wide contiguous vector loads/stores (`vec_ld`/`vec_st`) exclusively against its private 768 KB *Array Memory (AM)* at an aggregate internal bandwidth of ∼921.6 GB/s. Crucially, **no hardware cache exists between off-chip DDR4 and SM/AM**: all data movement must be explicitly orchestrated by software using point-to-point hardware DMA channels (`dma_p2p`), where each cluster's measured direct DDR4 DMA bandwidth is $\mathcal{B}_{\text{DDR}} = 16.22$ GB/s [9].

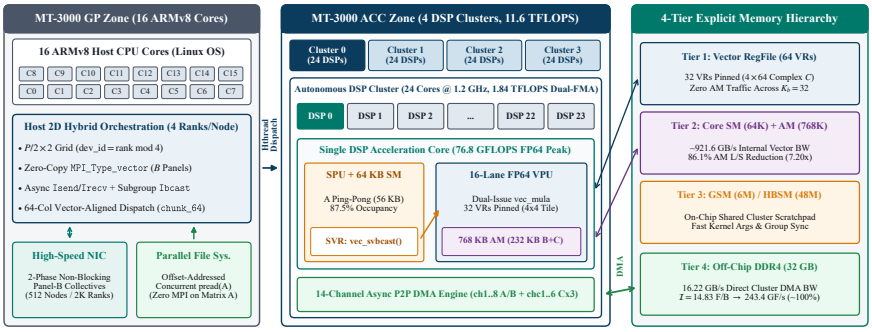


Fig. 1. MT-3000 multi-zone heterogeneous node architecture, four-tier software-managed explicit memory hierarchy, and mapping of our end-to-end hierarchical ZGEMM co-design across the 16-core ARMv8 GP Zone and four autonomous 24-core DSP clusters in the ACC Zone.

B. Split-Complex Planar Formulation vs. Algebraic 3M

Given complex matrices $A = A_{\mathrm{Re}} + iA_{\mathrm{Im}} \in \mathbb{C}^{M\times K}$, $B = B_{\mathrm{Re}} + iB_{\mathrm{Im}} \in \mathbb{C}^{K\times N}$, and $C = C_{\mathrm{Re}} + iC_{\mathrm{Im}} \in \mathbb{C}^{M\times N}$, ZGEMM computes:

$$\begin{aligned} C_{\mathrm{Re}} &\leftarrow C_{\mathrm{Re}} + A_{\mathrm{Re}}B_{\mathrm{Re}} - A_{\mathrm{Im}}B_{\mathrm{Im}}, \\ C_{\mathrm{Im}} &\leftarrow C_{\mathrm{Im}} + A_{\mathrm{Re}}B_{\mathrm{Im}} + A_{\mathrm{Im}}B_{\mathrm{Re}}. \end{aligned} \tag{2}$$

Conventional BLAS interfaces (`cblas_zgemm`) store complex numbers in interleaved Array-of-Structures (AoS) layout $([x_0, y_0, x_1, y_1, \dots])$. On 16-lane VPU architectures without zero-overhead de-interleaving load crossbars, AoS forces costly in-register shuffle permutations on every vector load. Conversely, storing real and imaginary components in *split-complex planar* (Structure-of-Arrays, SoA) format aligns 16 consecutive real or imaginary elements directly onto single 128-byte `vec_ld` transactions.

While Gauss/Karatsuba 3M methods reduce arithmetic count from $8MNK$ to $6MNK$ FLOPs via $(A_{\mathrm{Re}} + A_{\mathrm{Im}})(B_{\mathrm{Re}} + B_{\mathrm{Im}})$ [2], they are ill-suited to scratchpad-constrained DSP supercomputers for two reasons: (i) forming auxiliary matrix sums consumes extra DDR4 memory and DMA bandwidth, which—as shown below—is the true bottleneck rather than FMA throughput; and (ii) subtracting large intermediate products amplifies floating-point rounding error proportionally to $\max(\|A\|\|B\|/\|C\|)$ [12]. We therefore execute exact 4M accumulation in (2) via native dual-issue fused `vec_mula` instructions, preserving strict IEEE-754 FP64 accuracy without workspace allocation.

C. Machine Balance Diagnosis: The Scratchpad Memory Wall

Dividing the 24-core cluster vector peak $\mathcal{P}_{\mathrm{cluster}} = 1{,}843.2$ GFLOPS by its off-chip DDR4 DMA bandwidth $\mathcal{B}_{\mathrm{DDR}} = 16.22$ GB/s exposes a formidable *machine balance ridge point* [13]:

$$\mathcal{I}_{\mathrm{ridge}} = \frac{\mathcal{P}_{\mathrm{cluster}}}{\mathcal{B}_{\mathrm{DDR}}} = \frac{1{,}843.2\,\mathrm{GFLOPS}}{16.22\,\mathrm{GB/s}} = 113.64\,\mathrm{FLOPs/B}. \tag{3}$$

However, for a complex tile (M_b, K_b, N_b) buffered inside a core's 64 KB SM and 768 KB AM, every K_b step streams $16(M_bK_b + K_bN_b)$ bytes of A and B from DDR4 to perform $8M_bK_bN_b$ FLOPs, bounding the asymptotic tile arithmetic intensity to $\mathcal{I}_{\max} < \frac{M_bN_b}{2(M_b+N_b)}$. Even if A filled the entire 64 KB SM ($M_b \approx 64$), $\mathcal{I}_{\max} \le 16$ FLOPs/Byte $\ll 113.64$ FLOPs/Byte. Consequently, *multi-core ZGEMM on MT-3000 is fundamentally DDR4-DMA-bandwidth-bound at the cluster level, yet AM-port- and FMA-latency-bound within each core*. Achieving theoretical hardware limits requires: (1) maximizing tile arithmetic intensity within the 64 KB SM ceiling while achieving 100% overlap between DDR4 DMA transfers (including C tiles) and VPU execution; and (2) preventing MPI collectives and strong-scaling tile fragmentation from degrading cluster DMA efficiency at scale.

III. Hierarchical 2D Data Orchestration and Overlapped Collective Pipeline

A. Stationary-A, Panel-Broadcast-B 2D Process Grid

In a conventional 1D row-block baseline (`gemm_host_AS.c`), Rank 0 reads full matrices A and B from storage, scatters M/P rows of A to P ranks via `MPI_Scatterv`, and broadcasts 100% of B across `MPI_COMM_WORLD` via `MPI_Bcast`. At $P = 2{,}048$ ranks (512 nodes), global broadcast contention causes communication latency to explode to 102.25 s while shrinking per-rank row height to $M_{\mathrm{local}} = 8192/2048 = 4$ rows. Conversely, general $\sqrt{P} \times \sqrt{P}$ 2D SUMMA [14] requires $\sqrt{P}$ iterative broadcast-compute barriers over both A and B panels, repeatedly stalling bare-metal DSP group launches, whereas 2.5D/3D replication [15] incurs costly post-compute `MPI_Allreduce`/`Reduce_scatter` over C and exceeds the 32 GB per-node DDR4 capacity when four ranks share one node.

We design an asymmetric **Stationary-A, Panel-Broadcast-B 2D hybrid topology** organized as a $(P/G) \times G$ logical grid (Fig. 2(a) and Algorithm 1). Using cyclic subgroup assignment via `MPI_Comm_split`, each global rank $p \in [0, P-1]$ maps to column-panel subgroup $g = p \bmod G$, intra-group row index $r = \lfloor p/G \rfloor \in [0, P_g - 1]$ (where $P_g = P/G$), and physical node DSP cluster $d = p \bmod 4$. Matrix $B \in \mathbb{C}^{K\times N}$ is partitioned column-wise into G disjoint vertical panels $B_g \in \mathbb{C}^{K\times N_g}$ with $N_g = \lfloor N/G \rfloor + \mathbb{I}(g < N \bmod G)$, while matrix $A \in \mathbb{C}^{M\times K}$ is partitioned row-wise into P_g horizontal slices $A_r \in \mathbb{C}^{M_r\times K}$. Because subgroup leader ranks ($r = 0$) bear extra inter-leader dispatch overhead for B_g, we employ an asymmetric load-balanced row rule (`gemm_host_AB_opt6.c`) with

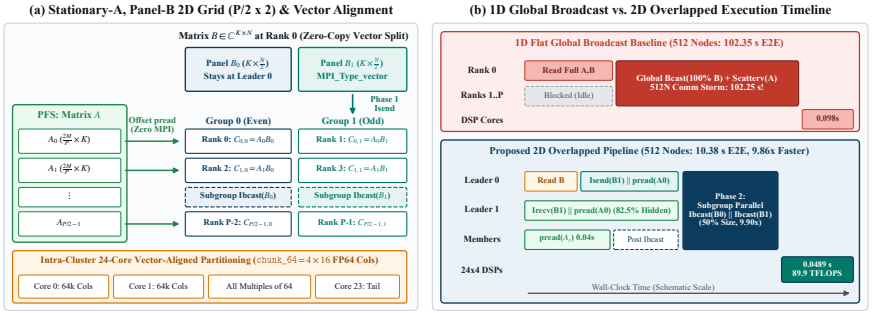


Fig. 2. Proposed Stationary-A, Panel-Broadcast-B 2D hybrid orchestration ($P/2 \times 2$): (a) zero-copy strided `MPI_Type_vector` dispatch of B panels, zero-communication offset-addressed parallel `pread` of A slices, disjoint $C_{r,g}$ ownership without reduction, and 64-column vector-aligned 24-core partitioning; (b) asynchronous execution timeline contrasting the 512-node 1D communication storm against our two-phase overlapped pipeline.

remainder $R_M = (M - M_0) \bmod (P_g - 1)$ that shields leaders from remainder rows:

$$M_r = \begin{cases} \lfloor M/P_g \rfloor, & r = 0, \\ \left\lfloor \dfrac{M - M_0}{P_g - 1} \right\rfloor + \mathbb{I}(r \le R_M), & 1 \le r < P_g. \end{cases} \tag{4}$$

Process (r, g) exclusively computes the disjoint output submatrix $C_{r,g} \leftarrow C_{r,g} + A_r B_g \in \mathbb{C}^{M_r \times N_g}$ directly inside DSP-mapped memory (`hthread_malloc`), requiring **zero network communication of A and zero post-compute reduction of C.**

B. Zero-Copy Strided Vector Dispatch and Two-Phase Collectives

Because B is stored in row-major order, column panels B_g ($g \ge 1$) consist of K non-contiguous segments of width N_g separated by stride N. Packing B_g into intermediate contiguous host buffers at Rank 0 would consume gigabytes of scarce node RAM and pollute memory bandwidth. Instead, Rank 0 constructs and commits a zero-copy derived strided datatype (`operator.c`):

$$\mathcal{D}_{B_g} \triangleq \texttt{MPI_Type_vector}(K,\ N_g,\ N,\ \texttt{MPI_DOUBLE}), \tag{5}$$

and dispatches B_g directly from the in-place column offset $\&B[0, j_{\text{start}}(g)]$ to each subgroup leader ($r = 0, g \ge 1$) via non-blocking `MPI_Isend`, which the receiving leader unpacks on-the-fly into a contiguous DSP-mapped buffer of layout (K, N_g) via `MPI_Irecv` (Phase 1). Immediately upon Phase-1 completion, all G subgroup leaders concurrently execute non-blocking `MPI_Ibcast` of their half-width contiguous panels B_g across isolated subgroup communicators $\mathcal{C}_g$ of size P/G (Phase 2).

Under the Rabenseifner binomial/scatter-allgather collective cost model [16] with startup latency α, per-byte transfer time β, and network contention penalty $\gamma(P)$ that grows superlinearly when switch buffers saturate at scale, 1D global broadcast versus our 2D subgroup broadcast satisfies:

$$\begin{aligned} T^{\text{1D}}_{\text{comm}}(P) &= \alpha \log_2 P + 32KN\left(\beta + \gamma(P)\right), \\ T^{\text{2D}}_{\text{comm}}(P, G) &= T_{\text{p2p}}(G) + \alpha \log_2 \left(\frac{P}{G}\right) \\ &\quad + \frac{32KN}{G}\left(\beta + \gamma\left(\frac{P}{G}\right)\right). \end{aligned} \tag{6}$$

Algorithm 1 Host-Side 2D Hybrid Orchestration & Overlapped Pipeline

Input: Dimensions (M, K, N), total ranks P, panel groups $G = 2$, rank p.
Output: Distributed disjoint submatrices $C_{r,g} = C_{r,g} + A_r B_g$ in DSP memory.

1: Bind rank to autonomous DSP cluster: $d \leftarrow p \bmod 4$; `hthread_dev_open`(d).
2: Split communicator: $g \leftarrow p \bmod G$, $r \leftarrow \lfloor p/G \rfloor$, $\mathcal{C}_g \leftarrow$ `MPI_Comm_split`(g, r).
3: Compute local dimensions (M_r, N_g) and row offset $i_0(r)$ via (4).
4: Allocate DSP-pinned buffers $A_r \in \mathbb{C}^{M_r \times K}, B_g \in \mathbb{C}^{K \times N_g}, C_{r,g} \in \mathbb{C}^{M_r \times N_g}$.
5: **if** $p = 0$ **then**
6: Load $B_{\text{Re}}, B_{\text{Im}}$; commit strided $\mathcal{D}_{B_{g'}}$ via (5); post `MPI_Isend` to leaders $g' \in [1, G-1]$; extract local B_0. ▷ *Phase-1 async send*
7: **else if** $r = 0$ **and** $g > 0$ **then**
8: Post non-blocking `MPI_Irecv` for $B_{\text{Re},g}, B_{\text{Im},g}$ from Rank 0.
9: **end if**
10: **Concurrently** execute offset-addressed `pread64` of $A_{\text{Re},r}, A_{\text{Im},r}$ at byte offset $8 \cdot i_0(r) \cdot K$ into `HT_MEM_RO` buffer A_r. ▷ *Overlaps Phase-1 NIC*
11: **if** $r = 0$ **then**
12: `MPI_Waitall` on Phase-1 handles; post `MPI_Ibcast`$(B_g, \mathcal{C}_g)$.
13: **else**
14: Post `MPI_Ibcast`$(B_g, \mathcal{C}_g)$ immediately after `pread64`. ▷ *Phase-2 tree*
15: **end if**
16: Compute 64-col vector-aligned 24-core partition $\{N_{\text{thread}}(t)\}_{t=0}^{23}$ via (7).
17: `MPI_Waitall` on Phase-2 `Ibcast`; launch 24-core DSP group (Algorithm 2).

Setting $G = 2$ simultaneously halves the broadcast payload (1.074 GB $\rightarrow$ 0.537 GB at $N = 8192$), halves the collective fan-out (2,048 $\rightarrow$ 1,024 ranks per isolated tree), and eliminates cross-tree switch contention, collapsing 512-node communication time by 9.90$\times$ (Section V-D). Table II summarizes the asymptotic advantages over 1D, SUMMA, and 2.5D schemes.

C. Concurrent Sliced `pread` and Communication-I/O Overlap

Rather than reading A at Rank 0 and scattering over MPI, every rank (r, g) computes its exact byte offset $\Delta_r = 8 \cdot i_0(r) \cdot K$ and invokes POSIX `pread64` (`read_data.c`) to stream its $M_r \times K$ slice directly from the parallel file system into Hthread DMA-mapped read-only memory (`HT_MEM_RO`). As shown in Fig. 2(b) and Lines 6–15 of Algorithm 1, subgroup leaders issue `pread64` *after* posting Phase-1 `MPI_Isend`/`MPI_Irecv` requests and *before* `MPI_Waitall`, overlapping host file-system DMA with RDMA network progress. Non-leader members ($r \ge 1$) read G/P of matrix A in parallel without root

TABLE II
ASYMPTOTIC PER-RANK COMPLEXITY COMPARISON ACROSS DISTRIBUTED GEMM TOPOLOGIES (P RANKS, SQUARE SCALE $M = N = K$, G PANELS)

Distributed Scheme	**Network Volume** $(A + B + C)$	**Collective Sync Stages**	**File I/O Critical Path**	**Local Rows** M_{local}	**DDR-Streaming Floor** T_{stream}
1D Flat Broadcast (AS)	$16\frac{N^2}{P} + 16N^2 \log_2 P + 0$	2 Global ($\log_2 P$)	$\mathcal{O}(N^2)$ Serialized at Root	N/P	$\geq 16N^2/\mathcal{B}_{\text{DDR}}$
2D SUMMA ($\sqrt{P} \times \sqrt{P}$) [14]	$\frac{32N^2}{\sqrt{P}} \log_2 \sqrt{P}$	$\frac{2N}{K_{\text{panel}}} \log_2 \sqrt{P}$ Iterative	$\mathcal{O}(N^2)$ or MPI-IO	$N/\sqrt{P}$	Iterative Launch Overhead
2.5D Replication (c copies) [15]	$\frac{32N^2}{\sqrt{Pc}} \log_2 \sqrt{\frac{P}{c}} + \frac{16N^2 \log_2 c}{P/c}$	$\sqrt{\frac{P}{c^3}}$ Bcast + Allreduce	$\mathcal{O}(N^2)$ + $c\times$ Mem Footprint	$N\sqrt{c/P}$	Extra C-Reduction Traffic
Proposed Stationary-A $(P/G \times G)$	$\mathbf{0} + \frac{\mathbf{16N^2}}{\mathbf{G}} \log_{\mathbf{2}}\left(\frac{\mathbf{P}}{\mathbf{G}}\right) + \mathbf{0}$	**1 Async Subgroup** ($\log_2 \frac{P}{G}$)	$\mathcal{O}\left(\frac{GN^2}{P}\right)$ **Overlapped `pread`**	$\mathbf{GN/P}$	$\geq \frac{\mathbf{16N^2}}{\mathbf{G} \cdot \mathcal{B}_{\text{DDR}}}$ ($G\times$ **Lower Floor**)

serialization and immediately post non-blocking `MPI_Ibcast` to join the subgroup tree.

D. 64-Column Vector-Aligned Partitioning and Optimality of $G = 2$

Within each cluster, N_g columns of B_g and $C_{r,g}$ are distributed across $T = 24$ DSP cores. Because our micro-kernel (Section IV) processes tiles of width $N_b = 64 = 4 \times 16$ columns, any core assigned a non-multiple of 64 columns suffers misaligned tail blocks requiring scalar SPU zero-padding loops (`N_cur != N_block`). We enforce a strict **64-column vector-aligned partitioning** (`chunk_64 = 64`): letting $Q = \lfloor N_g/64 \rfloor$, $q = \lfloor Q/24 \rfloor$, and $u = Q \bmod 24$, core $t \in [0, 23]$ receives:

$$N_{\text{thread}}(t) = \begin{cases} 64(q+1), & 0 \leq t < u, \\ 64q, & u \leq t < 23, \\ 64q + (N_g \bmod 64), & t = 23. \end{cases} \quad (7)$$

When $64 \mid N_g$, *all 24 DSP cores* execute exclusively on full 64-column vector tiles without scalar tails.

Equation (7) also yields a closed-form proof for why $G = 2$ **is the unique optimal panel group count** for $N = 8192$ on 24-core MT-3000 clusters:

- *Full 24-Core Active Occupancy Constraint:* Keeping all 24 cores active on ≥ 1 full 64-column block requires $N_g = N/G \geq 24 \times 64 = 1{,}536 \implies G \leq 5.33$. Any $G \geq 8$ ($N_g \leq 1{,}024 = 16 \times 64$) leaves at least 8 of 24 DSP cores (33.3%) completely idle.
- *Three-Stage C-Buffer Pipeline Constraint:* Sustaining cyclic triple-buffer rotation of C (Section IV) requires $N_{\text{thread}}(t) \geq 3N_b = 192$ columns ($Q \geq 72$) across the majority of cores. For $G = 4$, $N_g = 2{,}048$ ($Q = 32$), forcing 16 of 24 cores down to $N_{\text{thread}} = 64$ (a single tile), disabling outer C-DMA overlap.
- *Optimal Balance at $G = 2$:* $N_g = 4{,}096$ ($Q = 64$, $64 \mid N_g$) assigns 192 columns (3 tiles) to 16 cores and 128 columns (2 tiles) to 8 cores with zero tail padding, while doubling $M_{\text{local}} = 2M/P$ over $G = 1$ and halving both broadcast volume and the DDR-streaming floor.

IV. TRI-BUFFERED SCRATCHPAD PIPELINE AND 4×4 REGISTER-RESIDENT MICRO-KERNEL

A. 14-Channel Asynchronous DMA Tri-Buffered State Machine

On bare-metal DSP cores lacking hardware caches, conventional A/B ping-pong double buffering leaves the DDR4 read and write-back latencies of each $M_b \times N_b$ complex C tile (112 KB round-trip) unhidden between K-reduction loops. Worse, reusing only two C buffers creates a Read-After-Write / Write-After-Read hazard: at outer step j, the DMA engine must concurrently *write back* completed $C^{(j-1)}$ to DDR4, *accumulate* active $C^{(j)}$ on the VPU, and *prefetch* upcoming $C^{(j+1)}$ from DDR4.

We resolve this hazard via a **14-channel two-tier software-pipelined DMA state machine** (`matrix_mul_pipeline_opt` in `kernel_pipeline.c`; Fig. 3(a) and Algorithm 2):

1) **Middle-Tier C-Matrix Cyclic Triple Buffering (6 Channels, `chc1..chc6`):** Three split-complex buffer pairs $\{(\texttt{Cx_a}_s, \texttt{Cy_a}_s)\}_{s=1}^{3}$ reside in AM (168 KB total). Across column-tile steps $j = 0, N_b, 2N_b, \ldots$, cyclic rotation maps buffer $(j/N_b) \bmod 3$ to prefetch $C^{(j)}$ via strided P2P DMA (`dma_p2p`), buffer $(j/N_b - 1) \bmod 3$ to VPU K-accumulation, and buffer $(j/N_b - 2) \bmod 3$ to asynchronous DDR4 write-back of $C^{(j-2N_b)}$, hiding 100% of bidirectional C DDR4 traffic behind the $\lceil K/K_b \rceil$ inner FMA stages.
2) **Inner-Tier A/B Ping-Pong Double Buffering (8 Channels, `ch1..ch8`):** Along K, two complex A buffer pairs $\{(\texttt{Ax_s}_1, \texttt{Ay_s}_1), (\texttt{Ax_s}_2, \texttt{Ay_s}_2)\}$ reside in 64 KB SM (`ch1..ch4`, 56 KB), while two complex B buffer pairs $\{(\texttt{Bx_a}_1, \texttt{By_a}_1), (\texttt{Bx_a}_2, \texttt{By_a}_2)\}$ reside in 768 KB AM (`ch5..ch8`, 64 KB). Before entering the 4×4 micro-kernel on buffer pair $b \in \{1, 2\}$, the SPU posts non-blocking `dma_p2p` prefetches for step $k + K_b$ into alternate pair $3 - b$, hoisting all `dma_wait` barriers strictly *outside* the inner $K_b = 32$ FMA loop.

B. 4×4 Vector-Register-Resident Dual-FMA Micro-Kernel

Even after operands reside in SM and AM, naive loop ordering ($p \to i \to v$) loads and stores C_{Re} and C_{Im} vectors from AM at every $p \in [0, K_b - 1]$ step. For a 4×64 complex sub-tile over $K_b = 32$ steps, naive inner-loop accumulation issues $32 \times (8$ B-loads$+4 \times 8$ C-loads$+4 \times 8$ C-stores$) = 2{,}304$ AM vector memory instructions for 2,048 vector FMAs (0.89 FMAs per AM access), causing severe AM port contention.

To break this bottleneck, we design a 4×4 **outer-product register-resident micro-kernel** (`COMPUTE_K_BLOCK_OPT_C`, Fig. 3(b)):

- **Exact 32-VR Accumulator Pinning (Zero Spilling):** We unroll 4 rows along M_b and 4 vector segments ($4 \times 16 = 64$ complex columns) along N_b, allocating $4 \times 4 \times 2 = \mathbf{32}$ physical vector registers ($\mathbf{cx}_{u,v}, \mathbf{cy}_{u,v}$ for $u, v \in \{0, 1, 2, 3\}$) to hold the 4×64 complex

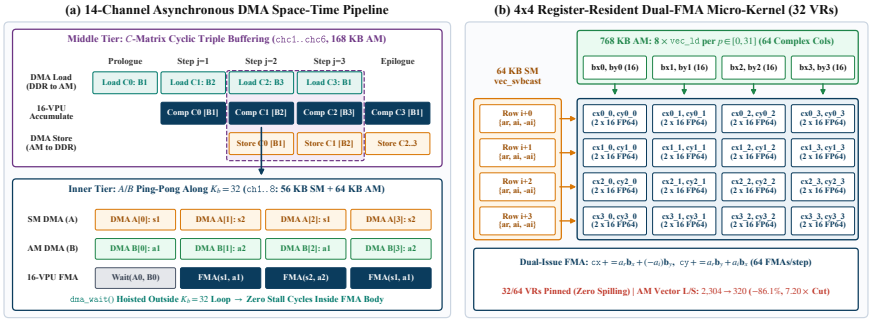


Fig. 3. Intra-chip scratchpad and register co-design: (a) 14-channel asynchronous P2P DMA space-time pipeline nesting inner-tier A/B ping-pong double buffering (`ch1..ch8`) inside middle-tier C-matrix cyclic triple buffering (`chc1..chc6`); (b) 4×4 vector-register-resident outer-product dual-FMA micro-kernel pinning 32 physical vector accumulators to eliminate 86.1% of Array Memory vector load/store traffic.

Algorithm 2 Device-Side 14-Channel Tri-Buffered DSP Kernel

Input: Submatrices $A \in \mathbb{C}^{M_r \times K}, B \in \mathbb{C}^{K \times N_g}, C \in \mathbb{C}^{M_r \times N_g}$; core ID $t \in [0, 23]$; tile $(M_b, K_b, N_b) = (56, 32, 64)$.
Output: Updated core slice $C[:, j_0(t) : j_0(t) + N_{\text{thread}}(t)]$.

Allocate SM ping-pong $\{Ax_{1,2}, Ay_{1,2}\}$ (56 KB) and AM buffers $\{Bx_{1,2}, By_{1,2}\}$ (64 KB) + cyclic triple buffers $\{Cx_{1..3}, Cy_{1..3}\}$ (168 KB).
for $i = 0$ **to** $M_r - 1$ **step** M_b **do**
  Prologue ($j = 0$)**:** Async `dma_p2p` load $C[i, j_0]$ into (Cx_1, Cy_1).
  for $j = N_b$ **to** $N_{\text{thread}}(t) - 1$ **step** N_b **do**
    Rotate $(s_{\text{store}}, s_{\text{comp}}, s_{\text{load}})$ mod 3; post async `dma_p2p` prefetch of $C[i, j_0 + j]$ into $(Cx_{s_{\text{load}}}, Cy_{s_{\text{load}}})$.
    Post async `dma_p2p` for $A[i, 0] \to (Ax_1, Ay_1)$ and $B[0, j_0 + j - N_b] \to (Bx_1, By_1)$; `dma_wait` on $C_{s_{\text{comp}}}$ and prior $C_{s_{\text{store}}}$.
    for $k_{\text{idx}} = 0$ **to** $\lceil K/K_b \rceil - 1$ **do**
      $b \leftarrow (k_{\text{idx}} \bmod 2) + 1$; $\bar{b} \leftarrow 3 - b$.
      if $k_{\text{idx}} + 1 < \lceil K/K_b \rceil$ **then**
        Post async `dma_p2p` prefetch $A_{k+K_b} \to (Ax_{\bar{b}}, Ay_{\bar{b}})$ (`ch1..4`) and $B_{k+K_b} \to (Bx_{\bar{b}}, By_{\bar{b}})$ (`ch5..8`).
      end if
      `dma_wait` on active (Ax_b, Ay_b, Bx_b, By_b).
      for $r_0 = 0$ **to** $M_{\text{cur}} - 1$ **step** 4 **do**
        Load 32 VRs $\{\mathbf{cx}_{u,v}, \mathbf{cy}_{u,v}\}_{u,v=0}^{3}$ from $(Cx_{s_{\text{comp}}}, Cy_{s_{\text{comp}}})$.
        for $p = 0$ **to** $K_{\text{cur}} - 1$ **do**
          Load 8 vectors $\{\mathbf{bx}_v, \mathbf{by}_v\}_{v=0}^{3}$ via `vec_ld` from (Bx_b, By_b).
          for $u = 0$ **to** 3 **do**
            Broadcast $\{a_r, a_i, -a_i\}$ via `vec_svbcast`; execute 8 dual-issue `vec_mula` updates (8).
          end for
        end for
        Store 32 VRs $\{\mathbf{cx}_{u,v}, \mathbf{cy}_{u,v}\}$ to $(Cx_{s_{\text{comp}}}, Cy_{s_{\text{comp}}})$.
      end for
    end for
    Post async `dma_p2p` store $(Cx_{s_{\text{comp}}}, Cy_{s_{\text{comp}}}) \to C[i, j_0 + j - N_b]$.
  end for
  Epilogue: Accumulate final j-tile and `dma_wait` final C write-backs.
end for

C sub-tile. Together with 8 streaming B vector registers ($\mathbf{bx}_{0..3}, \mathbf{by}_{0..3}$) and 3 scalar-broadcast A registers ($\tilde{\mathbf{a}}_r, \tilde{\mathbf{a}}_i, -\tilde{\mathbf{a}}_i$), total live register occupancy is 43 out of 64 physical VRs (67.2%), guaranteeing **zero compiler register spilling** to AM.

- **Hoisted SPU Negation for Pure Dual-Issue `vec_mula`:** Because $-A_{\text{Im}}B_{\text{Im}}$ in (2) involves subtraction, using separate vector multiply and subtract instructions would forfeit dual-issue FMA throughput. Instead, for each row $i + u$, the SPU computes $-a_i$ in scalar arithmetic at zero VPU cost and broadcasts the triplet $\{a_r, a_i, -a_i\}$ across all 16 lanes via `vec_svbcast`, enabling all 32 real and 32 imaginary updates per p-step to execute exclusively as fused dual-issue `vec_mula` instructions:

$$\begin{aligned} \mathbf{cx}_{u,v} &\leftarrow \texttt{vec_mula}(\tilde{\mathbf{a}}_{r,u},\ \mathbf{bx}_v,\ \mathbf{cx}_{u,v}), \\ \mathbf{cx}_{u,v} &\leftarrow \texttt{vec_mula}(-\tilde{\mathbf{a}}_{i,u},\ \mathbf{by}_v,\ \mathbf{cx}_{u,v}), \\ \mathbf{cy}_{u,v} &\leftarrow \texttt{vec_mula}(\tilde{\mathbf{a}}_{r,u},\ \mathbf{by}_v,\ \mathbf{cy}_{u,v}), \\ \mathbf{cy}_{u,v} &\leftarrow \texttt{vec_mula}(\tilde{\mathbf{a}}_{i,u},\ \mathbf{bx}_v,\ \mathbf{cy}_{u,v}). \end{aligned} \tag{8}$$

- 86.1% **AM Vector Traffic Elimination:** Across $K_b = 32$ steps, $\mathbf{cx}_{u,v}$ and $\mathbf{cy}_{u,v}$ are loaded once before $p = 0$ (32 `vec_ld`) and stored once after $p = 31$ (32 `vec_st`), while streaming $32 \times 8 = 256$ B-vectors. Total AM vector memory instructions drop from 2,304 to $32 + 256 + 32 =$ **320**—an 86.1% **reduction** (7.20× **fewer AM accesses**)—raising inner-loop intensity to $64/8 =$ **8.0** vector FMAs per vector load.

C. Scratchpad Capacity Proof and Roofline Optimality of $(56, 32, 64)$

Selecting (M_b, K_b, N_b) is governed by coupled hardware constraints over SM (64 KB) and AM (768 KB):

$$\mathcal{S}_{\text{SM}} = 2_{\text{buf}} \times 2_{\mathbb{C}} \times M_b K_b \times 8\ \text{B} \le 64\ \text{KB} - \Delta_{\text{stack}}, \tag{9a}$$
$$\mathcal{S}_{\text{AM}} = (4K_bN_b + 6M_bN_b) \times 8\ \text{B} \le 768\ \text{KB}, \tag{9b}$$
$$M_b \equiv 0 \pmod 4, \quad N_b \equiv 0 \pmod{64}, \tag{9c}$$

where $\Delta_{\text{stack}} \approx 8$ KB reserves SM space for SPU stack frames, DMA descriptors, and kernel control structures, and (9c) eliminates register-tile remainder branches. Fixing $N_b = 64$ (4 vector widths) and $K_b = 32$ (amortizing C register load/store overhead to $64/320 = 20\%$ of AM traffic while keeping SM footprint compact), (9a) restricts $M_b \le \frac{56 \times 1024}{32 \times 32} = 56$. Choosing $(M_b, K_b, N_b) = (\mathbf{56}, \mathbf{32}, \mathbf{64})$ yields:

- $\mathcal{S}_{\text{SM}} = 4 \times 56 \times 32 \times 8 =$ **56.0** KB (87.5% **of** 64 **KB SM**, exact 8.0 KB control headroom, and $56 = 14 \times 4$ exact 4-row micro-tiles with zero row remainder);
- $\mathcal{S}_{\text{AM}} = (4 \times 32 \times 64 + 6 \times 56 \times 64) \times 8 = 64.0 + 168.0 =$ **232.0** KB (30.2% of 768 KB AM).

For reduction length K, processing one (M_b, N_b) output tile executes $8M_bKN_b$ FLOPs while transferring $16K(M_b +$

$N_b) + 32M_bN_b$ bytes across the DDR4 DMA bus, yielding operational arithmetic intensity (at $K = 8192$):

$$\mathcal{I}_{\mathrm{DMA}} = \frac{8M_bKN_b}{16K(M_b+N_b)+32M_bN_b} = \mathbf{14.83}\,\text{FLOPs/B}, \tag{10}$$

(asymptotic limit 14.93 FLOPs/Byte). Multiplying by the measured cluster DDR4 DMA bandwidth $\mathcal{B}_{\mathrm{DDR}} = 16.22$ GB/s [9] defines the **theoretical 24-core cluster Roofline ceiling**:

$$\mathcal{P}^{\mathrm{cluster}}_{\mathrm{roof}} = \mathcal{B}_{\mathrm{DDR}} \times \mathcal{I}_{\mathrm{DMA}} = 16.22 \times 14.83 = \mathbf{240.5}\ \text{GFLOPS}. \tag{11}$$

D. Strong-Scaling Off-Chip DDR-Streaming Floor Theorem

Equation (10) assumes $M_{\mathrm{local}} \geq M_b = 56$. During strong scaling of fixed problem size (M, K, N) across P ranks with G panel subgroups, each rank holds $M_{\mathrm{local}}(P, G) = GM/P$ rows. Once $P > GM/M_b$ ($P > 146$ for 1D $G = 1$; $P > 292$ for 2D $G = 2$ at $M = 8192$), the effective tile height clamps to $M_{\mathrm{eff}} = M_{\mathrm{local}} < 56$, causing operational arithmetic intensity to collapse to:

$$\mathcal{I}_{\mathrm{eff}}(P, G) \approx \frac{M_{\mathrm{local}}N_b}{2\,(M_{\mathrm{local}} + N_b)}. \tag{12}$$

At 512 nodes ($P = 2{,}048$), 1D partitioning ($G = 1, M_{\mathrm{local}} = 4$) degrades to $\mathcal{I}_{\mathrm{eff}} = \mathbf{1.88}$ FLOPs/Byte, whereas our 2D hybrid topology ($G = 2, M_{\mathrm{local}} = 8$) sustains $\mathcal{I}_{\mathrm{eff}} = \mathbf{3.56}$ FLOPs/Byte (1.89× higher). Equivalently, because every cluster must stream its assigned $B_g \in \mathbb{C}^{K\times(N/G)}$ panel ($16KN/G$ bytes) from DDR4 at least once, kernel wall time obeys the fundamental **DDR-streaming lower bound** $T_{\mathrm{stream}}(G)$:

$$T_{\mathrm{kernel}}(P, G) \geq \frac{16K(N/G) + 16M_{\mathrm{local}}(K + 2N/G)}{\mathcal{B}_{\mathrm{DDR}}}. \tag{13}$$

As $P \to \infty$ ($M_{\mathrm{local}} \ll N/G$), $T_{\mathrm{stream}}(1)/T_{\mathrm{stream}}(2) \to \mathbf{2.00}\times$ (1.074 GB/16.22 GB/s $\approx$ 0.066 s pure-B floor plus pipeline prologue/epilogue), predicting the 2.01× kernel speedup (0.0982 s → 0.0489 s) measured at 512 nodes for $N = 8192$. Moreover, doubling problem dimension to $N = 16384$ at 512 nodes ($P = 2{,}048, G = 2$) raises M_{local} from 8 to 16 rows and $\mathcal{I}_{\mathrm{eff}}$ in (12) from 3.56 to **6.40** FLOPs/Byte, yielding an analytical 512-node Roofline prediction of $2{,}048 \times 16.22 \times 6.39 = \mathbf{212.3}$ TFLOPS that closely predicts our measured 218.79 TFLOPS (0.1608 s).

V. Experimental Evaluation

A. Experimental Setup and Methodology

We evaluate our framework on an MT-3000 supercomputer scaling from 1 to 512 compute nodes, harnessing up to **8,192 ARMv8 GP cores, 2,048 MPI processes, 2,048 autonomous DSP clusters, 49,152 bare-metal DSP cores, and 786,432 FP64 VPU SIMD lanes** (5.94 PFLOPS aggregate chip-level peak). Host code is compiled with `mpicc -O2 -fopenmp` linked against Hthread and OpenBLAS; DSP kernels use the vendor VLIW/SIMD C compiler with explicit `lvector` and `dma_p2p` intrinsics.

All benchmarks execute IEEE-754 FP64 complex arithmetic ($8MNK$ FLOPs) across four campaigns: (1) *intra-node comparisons* ($N \in [512, 16384]$) against ARMv8 OpenBLAS [17] under planar **`4x DGEMM`** and interleaved AoS **`1x ZGEMM`**; (2) *scratchpad DMA buffering ablation* (`gemm_host_ASR`) isolating single-buffered (`s_opt`), A/B double-buffered (`d_opt`), and 14-channel tri-buffered (`pipeline_opt`) kernels sharing the identical 4×4 32-VR micro-kernel across 1 core, 1 cluster (24C), and 1 node (96C, 4 ranks); (3) *multi-scale strong scaling* (1–512 nodes) at 8192^3 (4.398 TFLOPs) and extreme 16384^3 (35.184 TFLOPs); and (4) *1–512 node weak scaling* under constant per-node base workloads ($512^3 = 1.07$ GFLOPs, $1024^3 = 8.59$ GFLOPs, and $2048^3 = 68.72$ GFLOPs/node, reaching 35.184 TFLOPs at 512 nodes). Distributed times record maximum wall-clock latency across ranks via `MPI_Barrier` and `MPI_Wtime`.

B. Intra-Node Throughput and Accuracy vs. OpenBLAS

Table III and Fig. 4 report intra-node throughput and precision:

1) Single-Core Compute Density: One MT-3000 DSP core running our tri-buffered 4×4 micro-kernel sustains 44.16–49.19 **GFLOPS** across all scales, reaching 64.1% **of its** 76.8 **GFLOPS dual-FMA peak**. Because 1 core's bandwidth demand ($49.19/14.83 = 3.32$ GB/s) lies well below the cluster's 16.22 GB/s DDR4 DMA ceiling, 1-core execution is compute-bound, outperforming 1-core ARMv8 OpenBLAS by 6.12×–7.28× (`4x DGEMM`) and 5.95×–6.47× (`1x ZGEMM`).

2) Single-Cluster vs. Multi-Core CPU Scaling: Pairing 16 ARMv8 cores with 4 DSP clusters makes *4 ARMv8 cores* the exact cluster-equivalent host resource (1/4 node GP vs. 1/4 node ACC). One 24-core DSP cluster reaches 243.44 **GFLOPS** at $N = 8192$ (18.066 s), achieving a 9.33× **speedup** over 4-core OpenBLAS (26.08 GFLOPS). Even against *all 16 ARMv8 cores*, one DSP cluster (25% of ACC) beats 16-core `1x ZGEMM` (89.65 GFLOPS) by 2.72× (10.86× node-equivalent) and 16-core `4x DGEMM` (44.56 GFLOPS) by 5.46× at $N = 8192$ and 6.57× at $N = 16384$ (199.21 vs. 30.34 GFLOPS, where 16-core `4x DGEMM` thrashes LLC/NUMA and `1x ZGEMM` hits OOM from 12 GiB AoS buffers, while our 2×2 host-tiled planar schedule runs workspace-free).

3) IEEE-754 FP64 Numerical Accuracy: Across all 6.71×10^7 complex outputs at $N = 8192$ (and Tile 0 at $N = 16384$), maximum ℓ_∞ error against OpenBLAS scales as $\mathcal{O}(K\epsilon_{\mathrm{FP64}})$ (Fig. 4(c)), bounded within 7.28×10^{-12}–2.18×10^{-8} (well below FP32 $\epsilon_{\mathrm{mach}} = 1.19 \times 10^{-7}$), confirming exact fused FMA accumulation without 3M cancellation.

C. Scratchpad DMA Buffering Ablation and Roofline Validation

Table IV, Fig. 5, and Fig. 7(a) isolate asynchronous DMA pipelining gains and validate our Roofline model:

1) Single vs. Double vs. Tri-Buffering Ablation: With the 4×4 32-VR micro-kernel fixed, single-buffered `s_opt` serializes DDR4↔SM/AM DMA with VPU compute. On 1 DSP core, as N reaches 8192 and outer DDR4 stride penalties

TABLE III
INTRA-NODE PERFORMANCE (GFLOPS), SPEEDUP, AND FP64 NUMERICAL ACCURACY (ℓ_∞ ERROR) VS. MULTI-THREADED ARMV8 OPENBLAS

	Single-Core Compute Density (1 Core)				Cluster-Level Throughput (1 DSP Cluster [24C] vs. Multi-Core CPU)						FP64 Max Error (ℓ_∞)	
Scale N ($M=N=K$)	DSP 1C Ours	CPU 1C 4xDGEMM	CPU 1C 1xZGEMM	Speedup (4D / 1Z)	DSP 24C (1 Clust.)	CPU 4C 4xDGEMM	CPU 16C 4xDGEMM	CPU 16C 1xZGEMM	vs. 4C (Clust-Eq)	vs. 16C (4D / 1Z)	$\max\lvert\Delta C_{\mathrm{Re}}\rvert$ (Real Part)	$\max\lvert\Delta C_{\mathrm{Im}}\rvert$ (Imag Part)
512	**44.16**	7.22	7.10	6.12× / 6.22×	**80.98**	23.32	17.23	40.99	3.47×	4.70× / 1.98×	7.28×10^{-12}	1.13×10^{-10}
1,024	**45.77**	7.07	7.69	6.48× / 5.95×	**139.73**	28.38	51.88	82.29	4.92×	2.69× / 1.70×	1.73×10^{-11}	3.27×10^{-10}
2,048	**49.19**	6.80	7.63	7.24× / 6.45×	**145.96**	22.84	86.15	92.49	6.39×	1.69× / 1.58×	4.37×10^{-11}	9.75×10^{-10}
4,096	**48.39**	6.78	7.74	7.13× / 6.25×	**173.05**	24.41	92.28	85.51	7.09×	1.88× / 2.02×	7.46×10^{-11}	2.79×10^{-9}
8,192	**49.16**	6.76	7.60	**7.28**× / **6.47**×	**243.44**	26.08	44.56	89.65	**9.33**×	5.46× / **2.72**×	1.89×10^{-10}	7.57×10^{-9}
16,384†	**49.16**	7.20	OOM	6.83× / —	**199.21**	26.62	30.34	OOM	7.48×	**6.57**× / —	1.05×10^{-9}	2.18×10^{-8}

†At $N = 16384$, allocating 12 GiB of interleaved AoS buffers alongside planar host and Hthread DMA buffers exceeds 32 GB node DDR4; single-cluster evaluation executes via 2×2 host-tiled scheduling without AoS workspace.

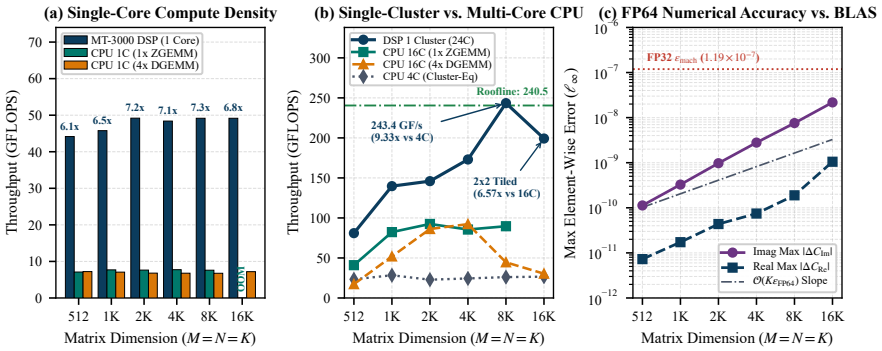


Fig. 4. Intra-node evaluation across $N = 512$ to 16384: (a) single-core throughput and speedup of 1 MT-3000 DSP core over 1-core ARMv8 OpenBLAS (`1x ZGEMM` and `4x DGEMM`); (b) 1 DSP cluster (24 cores, 1/4 node ACC Zone) vs. 4-core (cluster-equivalent) and full 16-core ARMv8 OpenBLAS relative to the 240.5 GFLOPS DDR4 DMA Roofline; (c) element-wise maximum absolute error (ℓ_∞) scaling linearly as $\mathcal{O}(K\epsilon_{\mathrm{FP64}})$.

TABLE IV
SCRATCHPAD DMA BUFFERING ABLATION (GFLOPS & ASYNC SPEEDUP VS. SINGLE-BUFFERED `s_opt`; IDENTICAL 4×4 32-VR MICRO-KERNEL)

	1 DSP Core				1 Cluster (24C)				1 Full Node (96C, 4 Ranks)			
N	s_opt	d_opt	tri	Gain	s_opt	d_opt	tri	Gain	s_opt	d_opt	tri	Gain
512	41.49	**44.29**	44.14	1.07×	71.47	72.37	**72.45**	1.03×	222.17	227.34	**228.85**	1.03×
1K	42.96	**45.95**	45.08	1.07×	123.02	**124.64**	124.60	1.01×	462.10	**467.17**	465.81	1.01×
2K	43.23	**49.18**	49.14	1.14×	130.27	133.15	**133.18**	1.02×	501.07	**512.10**	512.07	1.02×
4K	41.56	**48.65**	47.73	1.17×	155.59	**160.20**	159.81	1.03×	613.76	**632.00**	630.40	1.03×
8K	33.79	**48.05**	47.23	**1.42**×	196.53	**224.79**	223.96	**1.14**×	783.47	**895.84**	893.32	**1.14**×
16K‡	—	—	—	—	—	—	—	—	838.03	**956.17**	953.54	**1.14**×

‡At $N = 16384$, 4-rank 2×2 node partitioning (20 GiB planar footprint) fits within 32 GB DDR4, whereas 1-rank untiled allocation exceeds single-device mapping.

grow, `s_opt` drops from 43.23 to 33.79 GFLOPS (130.17 s), whereas asynchronous ping-pong overlap (`d_opt`) sustains 48.05 **GFLOPS** (91.52 s)—a 1.422× **speedup** (29.69% **wall-time cut**). Across 1 cluster (24C) and 1 full node (96C, 4 MPI ranks in a 2×2 grid), asynchronous DMA buffering yields a steady 1.141×–1.144× **speedup** (12.4%–12.6% time savings) at $N \geq 8192$, raising 96-core single-node throughput from 783.47 to 895.84 **GFLOPS** at $N = 8192$ and from 838.03 to 956.17 **GFLOPS** at $N = 16384$ (36.80 s vs. 41.98 s). On a quiet single node, `d_opt` and `pipeline_opt` agree within $< 0.9\%$ (956.17 vs. 953.54 GFLOPS) due to minor 14-channel modulo descriptor overhead, whereas at 512-node scale (Section V-E), `pipeline_opt`'s cyclic C-tile triple buffer provides decisive jitter resilience.

2) Empirical Roofline Saturation: In Fig. 5, as N grows from 512 (1 partial tile/core, no C-rotation, 80.98 GFLOPS) to 8192 (5–6 full 64-column tiles/core at $\mathcal{I}_{\mathrm{DMA}} = 14.83$ FLOPs/Byte), 24-core throughput climbs along the

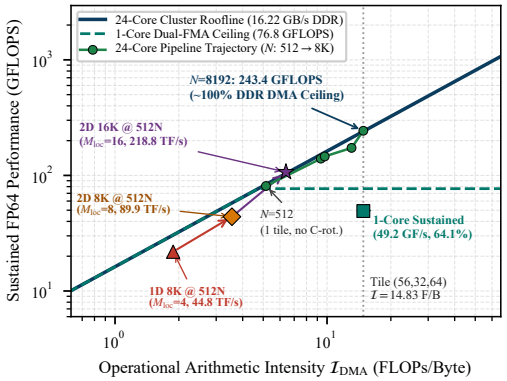


Fig. 5. Empirical Roofline validation on an MT-3000 DSP cluster (16.22 GB/s DDR4 DMA ceiling, 76.8 GFLOPS 1-core and 1,843.2 GFLOPS 24-core FMA peaks), showing 1-core efficiency (64.1%), 24-core saturation (243.44 GFLOPS), and 512-node strong-scaling operating points across 1D 8K (44.8 TF/s), 2D 8K (89.9 TF/s), and 2D 16K (218.8 TF/s).

16.22 GB/s DDR4 slope to 243.44 **GFLOPS**, matching our 240.5 GFLOPS ceiling in (11) within 1.2% ($\sim$100% DDR4 DMA saturation). Across 4 clusters per node, the theoretical node ceiling $4 \times 240.5 = \mathbf{962.0}$ GFLOPS matches our measured 96-core throughput of 956.17 **GFLOPS** within 0.6%, while all three 512-node operating points—1D 8K ($\mathcal{I} = 1.88, 44.78$ TF/s), 2D 8K ($\mathcal{I} = 3.56, 89.92$ TF/s), and 2D 16K ($\mathcal{I} = 6.40, 218.79$ TF/s)—align closely along the 2,048-cluster DMA Roofline.

D. Multi-Scale Strong Scaling: 8192^3 *and Extreme* 16384^3

Table V, Fig. 6, and Fig. 7(b) evaluate strong scaling across 1 to 512 nodes (96 to 49,152 DSP cores):

TABLE V
MULTI-SCALE DISTRIBUTED STRONG AND WEAK SCALING ACROSS 1 TO 512 MT-3000 NODES (4 TO 2,048 MPI RANKS; 96 TO 49,152 DSP CORES)

			8192^3 **Strong Scaling** (4.398 **TFLOPs): 1D Baseline vs. Proposed 2D Hybrid**								**Extreme** 16384^3 **2D Strong** (35.18 **TF**)			**Weak Scaling (`tri`)**	
Nodes	**MPI Ranks**	**DSP Cores**	**1D K. (s)**	**1D TF/s**	**1D Comm / E2E (s)**	**2D K. (s)**	**2D TF/s (Eff. %)**	**2D `pread` (s)**	**2D Comm / E2E (s)**	**2D/1D Gain (Kern / Comm)**	**16K K. (s)**	**16K TF/s (Eff. %)**	**`pread` / Comm (s)**[§]	2048^3 **TF/s (Eff. %)**[¶]	1024^3 / 512^3 **Eff. (%)**
1	4	96	4.5354	0.97	9.01 / 14.41	5.7812	0.76 (100%)	1.7782	2.89 / 10.45	0.78× / 3.12×	35.7795	0.98 (100%)	—	0.14 (100%)	100% / 100%
2	8	192	2.3148	1.90	13.94 / 16.50	2.8848	1.52 (100%)	1.2621	5.09 / 9.06	0.80× / 2.74×	15.8621	2.22 (**112.8%**)	—	0.28 (96.5%)	92.6% / 94.2%
4	16	384	1.2773	3.44	18.02 / 19.62	1.4777	2.98 (97.8%)	0.8289	7.10 / 9.38	0.86× / 2.54×	9.8105	3.59 (91.2%)	—	0.51 (89.8%)	88.4% / 98.9%
8	32	768	0.6532	6.73	18.88 / 19.58	0.7639	5.76 (94.6%)	0.3816	7.70 / 8.54	0.86× / 2.45×	5.0231	7.01 (89.0%)	2.19 / 39.58	1.06 (93.2%)	90.8% / 91.2%
16	64	1,536	0.3629	12.12	20.28 / 20.73	0.3877	11.34 (**93.2%**)	0.1992	9.73 / 10.16	0.94× / 2.08×	2.5957	13.56 (86.2%)	0.94 / 43.61	2.12 (93.1%)	83.6% / 90.8%
32	128	3,072	0.3319	13.25	24.26 / 24.69	0.2133	20.62 (84.7%)	0.1156	8.33 / 8.57	**1.56**× / 2.91×	1.3007	27.05 (**86.0%**)	0.53 / 45.29	4.22 (92.6%)	82.2% / 88.7%
64	256	6,144	0.1222	36.00	32.21 / 32.36	0.1284	34.26 (70.4%)	0.3060	10.26 / 10.45	0.95× / 3.14×	0.7112	49.47 (78.6%)	0.26 / 47.40	8.46 (92.8%)	77.0% / 54.0%
128	512	12,288	0.1017	43.24	26.96 / 30.10	0.0647	**68.01** (**69.8%**)	**0.0406**	8.75 / 8.85	**1.57**× / 3.08×	0.4223	83.31 (66.2%)	0.17 / 51.25	16.92 (92.8%)	89.5% / **87.5%**
256	1,024	24,576	0.0984	44.70	36.02 / 36.12	0.0538	**81.77** (42.0%)	4.0284	10.43 / 10.49	**1.83**× / 3.45×	0.2109	**166.82** (**66.3%**)	**0.11** / 54.53	33.76 (92.6%)	81.6% / 61.7%
512	**2,048**	**49,152**	0.0982	44.78	102.25 / 102.35	**0.0489**	**89.92** (23.1%)	3.0916	**10.32 / 10.38**	**2.01**× / **9.90**×	**0.1608**	**218.79** (43.5%)	0.12 / **47.34**	**67.24** (**92.2%**)	**80.1%** / 55.4%

[§] At 1–4 nodes ($M_{\text{local}} \geq 2048$), 16384^3 benchmarks use in-memory initialization because staging full 4.30 GB unpartitioned B at Rank 0 alongside 20 GiB node DSP buffers exceeds single-node host RAM; 8–512 nodes execute full storage-to-DSP `pread` + subgroup collectives. [¶] At 512 nodes (2048^3 weak scaling), tri-buffering (`pipeline_opt`) reaches 92.21% (0.5233 s, 67.24 TF/s) vs. double buffering's 89.64% (0.5382 s, 65.38 TF/s, +2.77% gain).

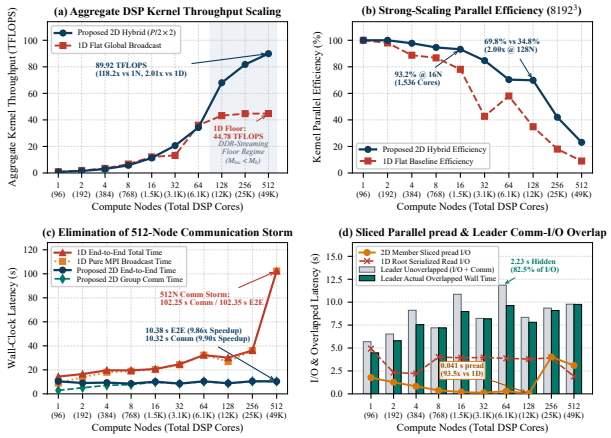


Fig. 6. Strong-scaling evaluation of 8192^3 complex GEMM across 1 to 512 MT-3000 supercomputer nodes (4 to 2,048 MPI ranks; 96 to 49,152 DSP cores): (a) aggregate DSP kernel throughput showing the 1D DDR-streaming floor (44.78 TFLOPS) vs. our 2D hybrid breakthrough (89.92 TFLOPS, 2.01×); (b) kernel strong-scaling parallel efficiency; (c) elimination of the 512-node 102.25 s MPI broadcast storm (9.90× comm speedup, 9.86× E2E speedup); (d) member sliced parallel `pread` latency (93.5× speedup at 128 nodes) and leader communication-I/O overlap (82.5% I/O hidden at 64 nodes).

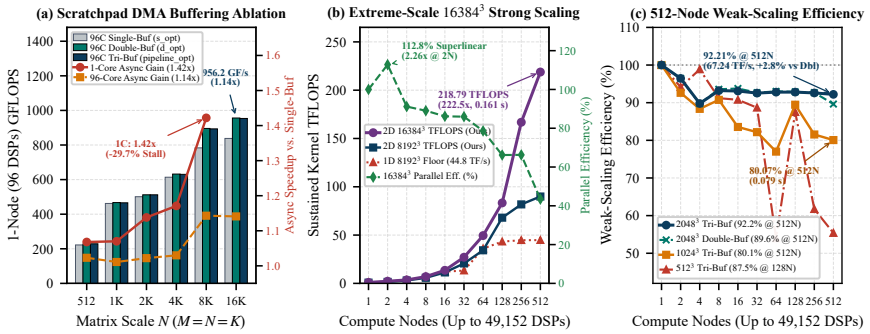


Fig. 7. Micro-architectural ablation and extreme-scale evaluation up to 512 nodes (49,152 DSP cores): (a) single/double/tri-buffered scratchpad DMA ablation reaching 956.17 GFLOPS/node and 1.42× 1-core async speedup; (b) extreme-scale 16384^3 (35.184 TFLOPs) vs. 8192^3 strong scaling achieving 218.79 TFLOPS (222.49× speedup) and 112.80% 2-node superlinear efficiency; (c) 1–512 node weak-scaling efficiency sustaining 92.21% (67.24 TFLOPS, +2.77% over double buffering) at 49,152 DSP cores.

1) 8192^3 **DDR-Floor Breakthrough and** 9.90× **Collective Storm Elimination:** At 1–16 nodes ($M_{\text{local}} \geq 128 > M_b$), our 2D scheme sustains 93.19% **parallel efficiency** (0.76 → 11.34 TFLOPS) and runs 1.38×–2.29× faster end-to-end than 1D. Beyond 64 nodes, 1D crosses $M_{\text{local}} < 32$ and stalls at the DDR4 streaming floor (13) (0.0982 **s,** 44.78 **TFLOPS,** 9.02% **efficiency** at 512 nodes), while its global broadcast detonates into a 102.25 **s storm** (102.35 s E2E). Doubling M_{local} (4 → 8) and halving B_g traffic enables our 2D co-design to sustain 69.84% **efficiency at 128 nodes** (68.01 **TFLOPS)** and reach 0.0489 **s (**89.92 **TFLOPS,** 118.20× **over 1 node)** at 512 nodes—an exact 2.01× **kernel speedup** validating (13)—while subgroup `MPI_Ibcast` holds 512-node communication flat at 10.32 **s (**9.90× **comm;** 10.38 **s E2E,** 9.86× **speedup).** Sliced parallel `pread` cuts member I/O to 0.0406 **s at 128 nodes** (93.5× **over 1D)** and hides 2.23 **s (**82.5%) of leader I/O behind Phase-1 transfers at 64 nodes (Fig. 6(d)).

2) Extreme-Scale 16384^3 **Strong Scaling to**

218.79 **TFLOPS:** Scaling $M = N = K = 16384$ (35.184 TFLOPs) across 1 to 512 nodes demonstrates two key phenomena (Table V, Fig. 7(b)). First, scaling from 1 node (35.7795 s, 0.983 TFLOPS, $M_{\text{local}} = 8192$) to 2 nodes (15.8621 s, 2.218 TFLOPS, $M_{\text{local}} = 4096$) yields a **superlinear** $2.256\times$ **speedup (**112.80% **efficiency)** as halving per-cluster $A_r/C_{r,g}$ footprint ($4.0 \rightarrow 2.0$ GiB) relieves 4-cluster shared DDR4 bank/page contention. Second, efficiency holds above 85.96% **through 32 nodes** (27.05 TFLOPS, $27.51\times$) and 66.26% **through 256 nodes** (0.2109 s, 166.82 **TFLOPS**, $169.64\times$). At 512 nodes (49,152 DSPs, $M_{\text{local}} = 16$), kernel runtime compresses to 0.1608 **s**, reaching 218.79 **TFLOPS** ($222.49\times$ **speedup**, 43.45% efficiency, $2.43\times$ over 8192^3). Across 8–512 nodes, subgroup broadcast of the 4.30 GB complex B matrix stays storm-free (39.58–47.34 s), while parallel `pread` reduces member I/O from 2.1881 s (8N) to 0.1117 **s** (256N) and 0.1235 **s** (512N, $37.9\times$ faster than leader I/O).

E. 512-Node Weak Scaling and Tri-Buffer Jitter Resilience

Table V and Fig. 7(c) evaluate 1–512 node weak scaling:

- 2048^3 **Base Workload (**68.72 **GFLOPs/node** $\rightarrow$ 35.184 **TFLOPs @ 512N):** Runtime stays nearly flat across a $512\times$ scale-out, rising by just 40.7 ms from 0.4825 s (1N) to 0.5233 **s at 512N** (67.24 TFLOPS) to sustain 92.21%**–**93.21% **efficiency** across 8–512 nodes (92.21% **at 49,152 DSPs**). While `d_opt` and `pipeline_opt` tie at 1 node (0.4824 s vs. 0.4825 s), at 512 nodes `d_opt` slips to 0.5382 s (89.64%, 65.38 TFLOPS), proving that cyclic C-tile triple buffering yields $+2.77\%$ **higher throughput (**$+2.57$ **percentage points)** by absorbing DDR/NIC jitter.
- 1024^3 **and Ultra-Fine** 512^3 **Workloads:** For sub-80 ms 1024^3 tasks (8.59 GFLOPs/node), `pipeline_opt` sustains 80.07% **efficiency at 512 nodes** ($0.0631 \rightarrow 0.0788$ s vs. `d_opt`'s 78.50%). Even for 13.8 ms 512^3 tasks (1.07 GFLOPs/node), `pipeline_opt` holds 87.51% **efficiency at 128 nodes** (12,288 DSPs; 55.39% vs. 52.22% at 512N).

VI. Related Work

Dense and Complex BLAS Micro-Kernels: CPU GEMM libraries—ATLAS [18], GotoBLAS [19], OpenBLAS [17], and BLIS [20]—rely on hierarchical cache-blocking and register-resident micro-kernels tailored to hardware caches, including 3M/4M complex GEMM in BLIS [2]. Unlike cache-coherent CPUs, MT-3000 DSP cores expose bare-metal 64 KB SM and 768 KB AM scratchpads without hardware cache refill, requiring explicit 14-channel DMA triple/double buffering alongside split-complex hoisted-negation `vec_svbcast` FMA scheduling.

Scratchpad Many-Core and MT-3000 Optimizations: Software-managed DMA double buffering was pioneered on IBM Cell BE [3], TI C6678 DSPs [4], and Sunway SW26010 [5], [6]. On MT-3000, Lu *et al.* [7] introduced the multi-zone architecture, Li *et al.* [21] ported molecular dynamics kernels, Fang *et al.* [8] developed `mtGEMM` for single-cluster real DGEMM, Mo *et al.* [10] studied irregular single-cluster GEMM, and Qi *et al.* [9], [11] explored single-cluster auto-tuning for real GEMM-like and attention operators. None address FP64 *complex* GEMM, C-matrix cyclic triple buffering, or *multi-node strong/weak scaling* across tens of thousands of DSPs.

Distributed GEMM and Collective Algorithms: ScaLAPACK [22], SUMMA [14], 2.5D GEMM [15], COSMA [23], and SLATE [24] optimize communication on homogeneous CPU/GPU grids, while MPICH [16] and hierarchical collectives [25] optimize tree topologies. Our Stationary-A, Panel-Broadcast-B 2D co-design uniquely couples zero-copy strided `MPI_Type_vector` subgroup collectives with parallel POSIX `pread` into DSP-pinned memory and 64-column SIMD alignment to suppress 2,048-rank storms and halve the DDR floor.

VII. Discussion and Conclusion

Portability Beyond $N = 16384$**:** Our co-design generalizes directly to broader multi-zone scratchpad accelerators (e.g., Sunway SW26010-Pro, multi-cluster DSPs, and spatial dataflow tiles). The constraint $N/G \geq 24 \times 3N_b = 4{,}608$ ($Q \geq 64$) governs optimal panel count G: while $G = 2$ is optimal at $N = 8192$ and reaches 218.79 TFLOPS at $N = 16384$, $N \geq 32768$ permits $G \in \{8, 16\}$, cutting subgroup broadcast payloads and the DDR floor (13) by $8\times$–$16\times$, while node-level shared-memory I/O leaders can curb Lustre contention above 1,024 readers.

Conclusion: We presented an end-to-end hierarchical co-design for distributed FP64 complex GEMM on MT-3000 supercomputers. Uniting a 14-channel asynchronous DMA tri-buffered scratchpad pipeline and a 4×4 32-VR dual-FMA micro-kernel (86.1% AM vector traffic cut) with Stationary-A, Panel-Broadcast-B 2D collectives and parallel `pread` achieves 49.19 GFLOPS/core ($1.42\times$ async DMA speedup; $6.12\times$–$7.28\times$ over 1-core OpenBLAS), saturates $\sim 100\%$ of the 240.5 GFLOPS cluster DDR4 Roofline (243.44 GFLOPS/cluster, 956.17 GFLOPS/node), cuts 512-node collective time by $9.90\times$ ($102.25 \rightarrow 10.32$ s), reaches 89.92 TFLOPS (8192^3) and 218.79 TFLOPS (16384^3, $222.49\times$ speedup), and sustains 92.21% weak-scaling efficiency across 512 nodes (49,152 DSPs).

REFERENCES

[1] R. F. Barrett, T. H. Chan, E. F. D'Azevedo, and E. C. Jaeger, "Complex version of high performance computing LINPACK benchmark (HPL)," *Concurrency Computat.: Pract. Exper.*, vol. 22, no. 5, pp. 573–587, Apr. 2010.

[2] F. G. Van Zee and T. M. Smith, "Implementing high-performance complex matrix multiplication via the 3M and 4M methods," *ACM Trans. Math. Softw.*, vol. 44, no. 1, pp. 7:1–7:36, Jul. 2017.

[3] J. A. Kahle, M. N. Day, H. P. Hofstee, C. R. Johns, T. R. Maeurer, and D. Shippy, "Introduction to the Cell multiprocessor," *IBM J. Res. Dev.*, vol. 49, no. 4/5, pp. 589–604, Jul. 2005.

[4] F. D. Igual, M. Ali, A. Friedmann, E. Stotzer, T. Wentz, and R. A. van de Geijn, "Unleashing the high-performance and low-power of multi-core DSPs for general-purpose HPC," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2012, pp. 1–11.

[5] H. Fu, J. Liao, J. Yang, L. Wang, Z. Song, X. Huang, C. Yang, W. Xue, F. Liu, F. Qiao, W. Zhao, X. Yin, C. Hou, C. Zhang, W. Ge, J. Zhang, Y. Wang, C. Zhou, and G. Yang, "The Sunway TaihuLight supercomputer: System and applications," *Sci. China Inf. Sci.*, vol. 59, no. 7, p. 072001, Jul. 2016.

[6] L. Jiang, C. Yang, Y. Ao, W. Yin, W. Ma, Q. Sun, and F. Liu, "Towards highly efficient DGEMM on the emerging SW26010 many-core processor," in *Proc. 46th Int. Conf. Parallel Process. (ICPP)*, Aug. 2017, pp. 422–431.

[7] K. Lu, Y. Wang, Y. Guo, C. Huang, S. Liu, R. Wang, J. Fang, T. Tang, Z. Chen, B. Liu, Z. Liu, Y. Lei, and H. Sun, "MT-3000: A heterogeneous multi-zone processor for HPC," *CCF Trans. High Perform. Comput.*, vol. 4, no. 2, pp. 150–164, 2022.

[8] J. Fang, K. Yu, P. Zhang, D. Dong, X. Qi, X. Hou, R. Wang, and K. Lu, "mtGEMM: An efficient GEMM library for modern multi-core DSPs," *IEEE Trans. Parallel Distrib. Syst.*, vol. 37, no. 4, pp. 905–919, Apr. 2026.

[9] X. Qi, J. Fang, P. Zhang, Y. Che, and J. Ren, "Constraint-driven auto-tuning of GEMM-like operators for MT-3000 many-core processor," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2025, pp. 185–199.

[10] H. Mo, Q. Wang, L. Liao, B. Li, L. Chi, and J. Liu, "Detailed analysis and optimization of irregular-shaped matrix multiplication on multi-core DSPs," in *Proc. 53rd Int. Conf. Parallel Process. (ICPP)*, Aug. 2024, pp. 719–729.

[11] X. Qi, J. Fang, P. Zhang, and Y. Che, "Optimizing attention for large language model inference on the MT-3000 many-core processor," *ACM Trans. Archit. Code Optim.*, vol. 23, no. 2, pp. 52:1–52:27, 2026.

[12] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[13] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[14] R. A. van de Geijn and J. Watts, "SUMMA: Scalable universal matrix multiplication algorithm," *Concurrency: Pract. Exper.*, vol. 9, no. 4, pp. 255–274, Apr. 1997.

[15] E. Solomonik and J. Demmel, "Communication-optimal parallel 2.5D matrix multiplication and LU factorization algorithms," in *Proc. 17th Int. Conf. Parallel Process. (Euro-Par), Part II*, Aug. 2011, pp. 90–109.

[16] R. Thakur, R. Rabenseifner, and W. Gropp, "Optimization of collective communication operations in MPICH," *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, pp. 49–66, Feb. 2005.

[17] X. Zhang, Q. Wang, and Y. Zhang, "Model-driven level 3 BLAS performance optimization on Loongson 3A processor," in *Proc. IEEE 18th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2012, pp. 684–691.

[18] R. C. Whaley, A. Petitet, and J. J. Dongarra, "Automated empirical optimizations of software and the ATLAS project," *Parallel Comput.*, vol. 27, no. 1–2, pp. 3–35, Jan. 2001.

[19] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, pp. 12:1–12:25, May 2008.

[20] F. G. Van Zee and R. A. van de Geijn, "BLIS: A framework for rapidly instantiating BLAS functionality," *ACM Trans. Math. Softw.*, vol. 41, no. 3, pp. 14:1–14:33, Jun. 2015.

[21] J. Li, H. Gu, J. Zhao, L. Qiao, C. Gong, and G. Zheng, "Transplantation and optimization of molecular dynamics simulation on MT-3000," *Future Gener. Comput. Syst.*, vol. 153, pp. 262–275, Apr. 2024.

[22] J. Choi, J. Demmel, I. Dhillon, J. Dongarra, S. Ostrouchov, A. Petitet, K. Stanley, D. Walker, and R. C. Whaley, "ScaLAPACK: A portable linear algebra library for distributed memory computers—design issues and performance," *Comput. Phys. Commun.*, vol. 97, no. 1–2, pp. 1–15, Aug. 1996.

[23] G. Kwasniewski, M. Kabić, M. Besta, J. VandeVondele, R. Solcà, and T. Hoefler, "Red-blue pebbling revisited: Near optimal parallel matrix-matrix multiplication," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2019, pp. 24:1–24:22.

[24] M. Gates, J. Kurzak, A. Charara, A. YarKhan, and J. Dongarra, "SLATE: Design of a modern distributed and accelerated linear algebra library," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Nov. 2019, pp. 26:1–26:18.

[25] B. S. Parsons and V. S. Pai, "Accelerating MPI collective communications through hierarchical algorithms without sacrificing inter-node communication flexibility," in *Proc. IEEE 28th Int. Parallel Distrib. Process. Symp. (IPDPS)*, May 2014, pp. 237–246.